`scripts/autonomous-qa/lib-jackett.sh` — User Guide

Last verified: 2026-07-26 (LVA-018 script-docs backfill cycle) **Inheritance:** HelixConstitution §11.4.18 (script docs); Lava §6.B ("Up" is not healthy), §6.H (no secret echo), §6.J (fail loudly), §6.R (no-hardcoding), §6.U (no sudo) **Classification:** project-specific

Overview

Sourceable library that brings up / tears down the **Jackett (Torznab) + FlareSolverr (Cloudflare-solver) sidecar** so lava-api-go can serve Cloudflare-protected tracker searches (notably IPTorrents) in the Phase-2 autonomous-QA matrix.

It is a thin autonomous-QA wrapper around the SAME mechanism the project's canonical validator (`scripts/validate-jackett-sidecar.sh`) uses: rootless podman (preferred) or docker — NO sudo — driving `tools/lava-containers/docker-compose.jackett.yml` with `--profile jackett --profile cloudflare`. Jackett/FlareSolverr ports bind to host **loopback only**; lava-api-go (host-net) reaches Jackett at `http://127.0.0.1:9117` and holds the api_key server-side. The Android app NEVER talks to Jackett (§6.H).

Raw `<runtime> compose` is correct here (the fragment is bridge + loopback, no `network_mode: host`); `./start.sh` + `BUILDAH_FORMAT=docker` are only needed for the host-net api-go profile whose image is locally built — the Jackett / FlareSolverr images are pulled with HEALTHCHECKs baked in.

Functions

Function	Purpose
`jackett_up`	Bring the sidecar up and BLOCK until the real Torznab surface answers, or fail loudly on timeout
`jackett_down`	Tear the sidecar down (both profiles, idempotent)

Usage

```
source scripts/autonomous-qa/lib-jackett.sh
jackett_up      # bring up jackett + flaresolverr, block until
        ready
# ... run Phase-2 IPTorrents searches against lava-api-go ...
jackett_down    # tear it all down
```

Env knobs (all optional)

Variable	Default	Meaning
`LAVA_QA_CONTAINER_RUNTIME`	auto-detect (podman → docker)	Force `podman` or `docker`
`LAVA_QA_JACKETT_CLOUDFLARE`	`1`	`0` = jackett only (skip the RAM-heavy FlareSolverr)
`LAVA_QA_JACKETT_TIMEOUT`	`120`	Readiness timeout, seconds

Every host/port/api_key/image/indexer comes from `.env` (placeholders in `.env.example`) per §6.R — `LAVA_API_JACKETT_APIKEY`, `LAVA_API_JACKETT_DEFAULT_INDEXER`, `LAVA_JACKETT_HOST_PORT`, `LAVA_FLARESOLVERR_HOST_PORT`, `LAVA_JACKETT_BIND_HOST`, …

Readiness contract (anti-bluff §6.B / §6.J)

`jackett_up` does NOT assert `container State==running`; it polls the REAL user-visible surfaces until they answer, and FAILS LOUDLY (with logs) on timeout:

- **Jackett** — `GET .../results/torznab/api?t=caps&apikey=<key>` must return HTTP 200 with parseable Torznab XML; an auth-error body (`incorrect api key`) is rejected explicitly. The api_key is REDACTED in every log line (§6.H). Without a valid key in `.env` the function refuses to run — an unauth probe would prove nothing.
- **FlareSolverr** (cloudflare profile only) — `POST /v1 {"cmd":"sessions.list"}` → 200 `{"status":"ok"}` (Chromium warm-up is slow; this is the de-facto liveness probe — FlareSolverr has no `GET /health`).

Return: `0` = sidecar READY (probed); non-zero = failed (container logs dumped, sidecar torn down).

Known unconfirmed value

The 120 s readiness default is **UNCONFIRMED** against the target gate host — the compose fragment itself notes the .NET/ Chromium cold-start seconds are placeholders. Measure real cold-boot on the Linux x86_64 gate host and tune `LAVA_QA_JACKETT_TIMEOUT` once observed; do not assume.

Companion files

- `scripts/validate-jackett-sidecar.sh` (+ its doc) — the canonical validator
- `tools/lava-containers/docker-compose.jackett.yml` — the compose fragment
- `docs/qa/jackett-sidecar-deployment.md` — sidecar deployment guide